

JavaScript Array and Loops

Introduction to Arrays

- In JavaScript, an **array** is a data structure used to store a collection of values in a single variable
- Arrays can hold multiple data types (numbers, strings, objects, or even other arrays)
- Array are zero-indexed, meaning the first element has an index of 0.
- Creating Arrays:

```
// Using square brackets
let numbers = [1, 2, 3, 4];

// Using the Array constructor
let fruits = new Array('Apple', 'Banana', 'Cherry');
```

Accessing Array Elements

- Accessing Array Elements

```
let colors = ['Red', 'Green', 'Blue'];  
console.log(colors[0]); // Output: 'Red'  
console.log(colors[2]); // Output: 'Blue'
```

- Modifying Arrays

```
let animals = ['Dog', 'Cat', 'Bird'];  
animals[1] = 'Fish'; // Modifies the second element  
console.log(animals); // Output: ['Dog', 'Fish', 'Bird']
```

Common Array Methods

- **Adding/Removing/Searching Elements**
 - Includes(): Searching elements
 - push(element): Adds to the end of the array.
 - pop(): Removes the last element.
 - unshift(element): Adds to the beginning.
 - shift(): Removes the first element.

Searching in Array

- Use `includes()` method to search a specific element.
- It returns a boolean (true or false)
- Syntax:

```
array.includes(searchElement, fromIndex)
```

- **Parameters**

- **searchElement**: The element to search for in the array.
- **fromIndex** (*optional*): The index to start searching from.
 - Defaults to 0 if omitted.
 - If negative, it counts from the end of the array.

Examples

- Basic Example

```
let fruits = ['apple', 'banana', 'cherry'];  
console.log(fruits.includes('banana')); // true  
console.log(fruits.includes('grape')); // false
```

- **Case Sensitivity** The method is case-sensitive:

```
let words = ['Hello', 'world'];  
console.log(words.includes('hello')); // false
```

Includes(): Using fromIndex

```
let numbers = [1, 2, 3, 4, 5];  
console.log(numbers.includes(3, 2)); // true (starts search at index 2)  
console.log(numbers.includes(3, 3)); // false (starts search at index 3)  
console.log(numbers.includes(3, -3)); // true (starts search 3 from the end)
```

Adding and Removing Elements to Arrays

- Use `push()` to add elements to the end of an array.
- Use `unshift()` to add elements to the beginning.
- Example:

```
let arr = [1, 2, 3];  
arr.push(4); // [1, 2, 3, 4]  
arr.pop();   // [1, 2, 3]  
arr.unshift(0); // [0, 1, 2, 3]  
arr.shift();  // [1, 2, 3]
```

Challenges

- Random Array Elements challenge

Using While Loops

- A **while** loop in JavaScript repeatedly executes a block of code as long as a specified condition is true
- Syntax:

```
while (condition) {  
    // code to execute  
}
```

While Loops: How It Works:

- The **condition** is evaluated before each iteration.
- If the condition is **true**, the code inside the loop runs.
- This process continues until the condition becomes **false**.

Example 1: Counting from 1 to 5

```
let count = 1;

while (count <= 5) {
  console.log(count); // Output: 1 2 3 4 5
  count++;
}
```

Example 2: User Input Validation

```
let password = "";

while (password !== "secret") {
    password = prompt("Enter the correct password:");
}

console.log("Access granted!");
```

Key Points:

- **Infinite Loop Risk:** Be careful to ensure the condition will eventually become false, or you might create an infinite loop.
- **Break Statement:** Use `break;` to exit the loop early if needed

Using For Loops

- In JavaScript, **for loops** are used to run a block of code repeatedly based on a specified condition. They are useful when you know how many times you want the loop to run.
- Syntax of a For Loop

```
for (initialization; condition; increment) {  
    // Code to be executed  
}
```

- **Explanation:**
 - **Initialization:** Sets a starting value (usually a variable). It runs once before the loop starts.
 - **Condition:** Specifies the condition for running the loop. If the condition is true, the loop continues; if false, it stops.
 - **Increment/Decrement:** Updates the loop variable after each iteration.

Example 1: Counting Up

```
for (let i = 1; i <= 5; i++) {  
    console.log("Count:", i);  
}
```

- Output:

```
Count: 1  
Count: 2  
Count: 3  
Count: 4  
Count: 5
```

Example 3: Looping Through an Array

```
let fruits = ["Apple", "Banana", "Cherry"];  
for (let i = 0; i < fruits.length; i++) {  
    console.log(fruits[i]);  
}
```

- Output:

```
Apple  
Banana  
Cherry
```

Break and Continue Statements

- **break:** Stops the loop immediately.
- **continue:** Skips the current iteration and moves to the next.

Example

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) {  
    continue; // Skip the number 3  
  }  
  console.log(i);  
}
```

- Output:

```
1  
2  
4  
5
```

Challenges

- The Fibonacci Exercise